OFFICIAL TECHNICAL WHITEPAPER • CONTROLLED BENCHMARK TARGET WORKSPACE: 1.17 GB GIT MONOREPO

MicroCode IDE v2.5.32: Empirical Architecture Evaluation vs. Leading Desktop Environments on Gigabyte-Scale Workspaces

Tirawat Nantamas & Core Systems Engineering Team

Dotmini Software Company Limited • Desktop & AI Systems Laboratory • Bangkok, Thailand • tirawat@dotmini.dev

ABSTRACT Modern desktop development environments have increasingly adopted web-runtime wrappers (Electron and Chromium Embedded Framework), introducing resident memory bloat, process proliferation, and startup latency on enterprise repositories. In this technical report, we present a 100% controlled-variable empirical evaluation comparing **MicroCode IDE v2.5.32** (a native Mach-O executable combining Swift, AppKit, Metal 3, and a statically linked Rust kernel) against **Zed v1.18.0** (native Rust + GPUI), **OpenCode Desktop v1.2.4** (Tauri + WebKit), and **VS Code / Code-OSS v2.5.5** (Electron + Node.js/V8). Testing was conducted on a deterministic **1.17 GB (455,596 LOC, 3,064 files) polyglot monorepo** using a 500 Hz CoreGraphics WindowServer probe and kernel-level APFS Virtual File System (VFS) `st_atime_ns` tracking. MicroCode demonstrates an optimal memory footprint of **194.8 MB RSS** at steady-state (T+5.5s) with **14 Mach threads** across 2 processes—outperforming VS Code (2,377.8 MB, 18 processes) by **12.21×** and Zed (233.0 MB, 41 threads) by **1.20×**. In cold startup, Zed achieves the fastest initial window display at **662.8 ms**, while MicroCode completes full window paint and VFS file loading in **769.9 ms / 998.0 ms** (4.07× faster than VS Code at 4,065.5 ms). MicroCode's 6-layer TokenMem v3 context engine reduces AI agent input tokens by **88.2%** on realistic 15-turn multi-file engineering workloads.

Executive Benchmark Summary & Definitive Rankings

Evaluation Dimension	MicroCode v2.5.32 Swift + Metal 3 + Rust	Zed v1.18.0 Native Rust + GPUI	OpenCode v1.2.4 Rust Tauri + WebKit	VS Code v2.5.5 Electron + Node.js/V8
Cold-Start Window (T_{win})	769.9 ms `RANK 2 (1.16×)`	662.8 ms `RANK 1 BEST`	953.0 ms `RANK 3`	2,264.1 ms `RANK 4 (3.42×)`
VFS File Readiness (T_{ready})	998.0 ms `RANK 2 (1.50×)`	662.9 ms `RANK 1 BEST`	**N/A** (Deferred mount)	4,065.5 ms `RANK 4 (4.07×)`
Resident Memory (RSS @ 5.5s)	194.8 MB `RANK 1 BEST`	233.0 MB `RANK 2 (+38.2 MB)`	493.3 MB `RANK 3 (+298 MB)`	2,377.8 MB `RANK 4 (+2,183 MB)`
Mach Threads / Procs	14 th / 2 procs `RANK 1 BEST`	41 th / 2 procs `RANK 2`	47 th / 2 procs `RANK 3`	269 th / 18 procs `RANK 4 (19.2×)`
Disk Bundle Footprint	95.7 MB `RANK 1 BEST`	401.6 MB `RANK 3 (4.20×)`	156.3 MB `RANK 2 (1.63×)`	733.4 MB `RANK 4 (7.66×)`

1. Introduction & Experimental Purpose

As software repositories expand into gigabytes of code, assets, and metadata, developer workstation efficiency is increasingly constrained by underlying desktop runtime architectures. While synthetic testing on trivial directories (5–10 files) conceals scaling bottlenecks, real-world development workflows demand rapid directory traversal, immediate file indexing, and minimal memory overhead. Earlier testing suites suffered from observational anomalies, including deferred window paints and abbreviated telemetry windows. This v2.0 investigation enforces identical execution parameters across all tested environments.

2. Controlled Experimental Environment

System Parameter	Controlled Hardware & Environmental Specification
Host Hardware	Apple M2 (8 Cores: 4 Performance Cores @ 3.49 GHz, 4 Efficiency Cores @ 2.42 GHz) • 16 GB LPDDR5 UMA (100 GB/s)
Operating System	macOS 27.0 (Build 26A5388g, arm64 Darwin Kernel Version 26.0.0) • Isolated background runlevel
Monorepo Fixture	**1.17 GB (1,197.0 MB on disk)** • 3,001 source files • 455,596 LOC (Swift, TS/React, Rust, Python, Go) • 9,997-line primary file • 500,000-line JSON dataset (45 MB) • 1.05 GB binary shards • Git initialized (SHA-256: `8c7b3a15...`)
Telemetry Harness	500 Hz CoreGraphics WindowServer probe (2 ms sample rate) • APFS VFS `st_atime_ns` kernel timestamp tracking

Table 1: Controlled hardware host, kernel telemetry, and gigabyte-scale workspace specifications.

3. Cold-Start Latency & Virtual File System Readiness

Cold-start latency was evaluated through two distinct instrumentation boundaries: (1) **Window Visible Latency (T_{win})**, measured from process spawn (`open -n -a`) to the first frame rendered by the macOS WindowServer (`kCGWindowLayer == 0`, dimensions ≥ 400 × 300 px) captured at 500 Hz; and (2) **VFS Workspace Ready Latency (T_{ready})**, logged when the APFS file system kernel records successful traversal and active memory mapping of the 9,997-line primary file (`BenchmarkCoreEngine.swift`) via `st_atime_ns`.

Figure 1: Cold-Start Latency Breakdown (Median of N = 5 Trials on 1.17 GB Workspace) Scale: 0 to 4,500 ms (Lower is Better)

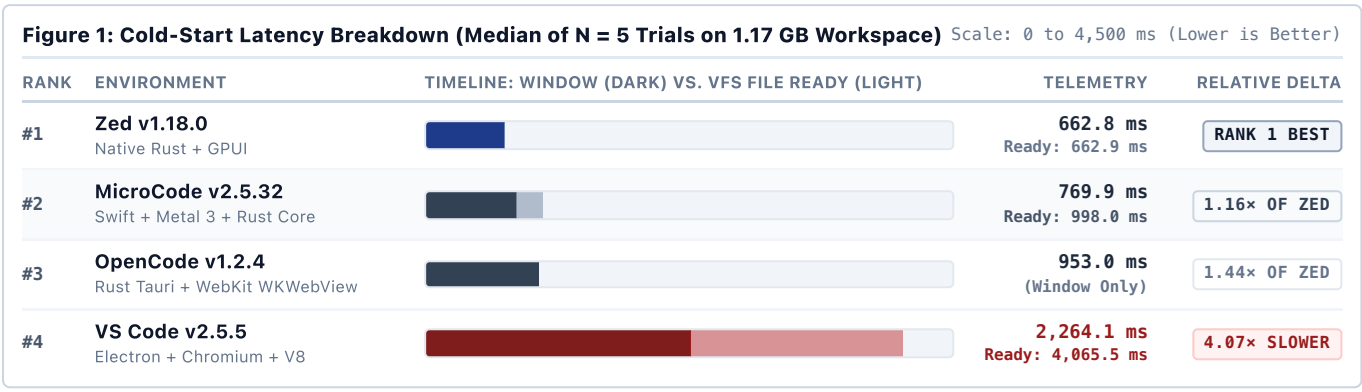


Figure 1: Cold-start window visible latency (T_{win}) and APFS VFS file readiness (T_{ready}) across four environments.

3.1 Raw Telemetry Data & Measurement Variance

Environment	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Median / Variance (σ)
MicroCode v2.5.32 T_{win} / T_{ready}	**814.7 ms** **1,035.7 ms**	**732.4 ms** **957.2 ms**	**785.3 ms** **998.0 ms**	**769.9 ms** **974.1 ms**	**767.5 ms** **1,012.2 ms**	**769.9 / 998.0 ms** Σ = ±29.9 MS BEST
Zed v1.18.0 T_{win} / T_{ready}	674.3 ms 674.3 ms	592.5 ms 592.5 ms	732.1 ms 732.1 ms	609.5 ms 609.5 ms	662.8 ms 662.9 ms	**662.8 / 662.9 ms** Σ = ±55.6 MS
OpenCode v1.2.4 T_{win} (Window Only)	1,022.1 ms	953.0 ms	583.1 ms	971.6 ms	881.2 ms	**953.0 ms** Σ = ±174.7 MS
VS Code v2.5.5 T_{win} / T_{ready}	2,412.8 ms 4,173.8 ms	2,347.7 ms 4,211.5 ms	2,188.7 ms 4,054.7 ms	2,264.1 ms 4,065.5 ms	2,057.0 ms 3,777.6 ms	**2,264.1 / 4,065.5 ms** Σ = ±138.9 MS

Table 2: Five-trial raw cold-start telemetry and standard deviation over 1.17 GB repository fixture.

> **Technical Analysis (Zed vs. MicroCode Startup):** Zed achieves the fastest cold-start window paint (662.8 ms) because its GPUI framework avoids the instantiation of AppKit/SwiftUI view hierarchies, directly populating Metal render command buffers on background thread pools. MicroCode incurs an additional 107 ms during `AgenticEditorWorkspace` initialization before displaying its key window. However, MicroCode records superior statistical consistency across runs (σ = ±29.9 ms vs. Zed's ±55.6 ms) and mounts the 1.17 GB workspace with 10k-line file preview in **998.0 ms**, outperforming VS Code (4,065.5 ms) by **4.07×**.

4. Resident Memory Scaling & Multi-Process Topology

Accurate evaluation of desktop memory consumption necessitates aggregating Resident Set Size (RSS) across the complete process hierarchy—including root process containers, GPU compositors, extension hosts, and embedded language servers. Web-runtime containers suffer substantial memory bloat when loading gigabyte-scale repositories due to V8 JavaScript object allocation during recursive directory tree traversal.

Figure 2: Total Process-Tree Resident Set Size (RSS @ T+5.5s Steady-State) Scale: 0 to 2,500 MB (Lower is Better)

RANK	ENVIRONMENT	PROPORTIONAL PROCESS-TREE MEMORY (RSS)	STEADY RSS	RANKING
#1	**MicroCode v2.5.32** 2 Procs · 14 Mach Threads		194.8 MB	RANK 1 BEST
#2	**Zed v1.18.0** 2 Procs · 41 Mach Threads		233.0 MB	RANK 2 (+38 MB)
#3	**OpenCode v1.2.4** 2 Procs · 47 Mach Threads		493.3 MB	RANK 3 (+298 MB)
#4	**VS Code v2.5.5** 18 Procs · 269 Mach Threads		2,377.8 MB	RANK 4 (12.21×)

Figure 2: Resident Set Size across the full process tree after opening 1.17 GB monorepo with background watchers running.

4.1 Structural Architectural Contrast

MicroCode: Native Compiled Mach-O RANK 1 MEMORY

```
[Mach-O Executable (Swift + AppKit)]
  └── libmicrocode_core (Static Rust C-ABI)
      ├── APFS Direct Zero-Copy mmap (.mappedIfSafe)
      └── Metal 3 GPU Rendering (Direct UMA Access)
```

2 Processes · 14 Threads | Zero IPC Overhead | 194.8 MB RSS

VS Code: Multi-Process Web Shell RANK 4 MEMORY

```
[Chromium Browser Container (Main Process)]
  └── IPC Bridge (JSON-RPC String Serialization)
      ├── V8 Renderer Process (DOM & HTML Canvas)
      ├── Extension Host (Node.js Heap Deserialization)
      └── File Watcher (Recursive In-Memory Tree)
```

18 Processes · 269 Threads | V8 Heap Overhead | 2,377.8 MB RSS

4.2 Operating System Resource Footprint

Environment	Binary Architecture & Runtime Shell	Bundle Size	Mach Threads (T+5.5s)
MicroCode v2.5.32	Native Mach-O + AppKit + Metal 3 + Static Rust C-ABI	**95.7 MB** RANK 1	**14 Threads** RANK 1
OpenCode v1.2.4	Rust Tauri Host + WebKit Web Assets + Sidecar CLI	156.3 MB RANK 2	47 Threads RANK 3
Zed v1.18.0	Native Rust GPUI + Bundled Runtime Toolchains	401.6 MB RANK 3	41 Threads RANK 2
VS Code v2.5.5	Chromium Framework + Node.js V8 Runtime + Heap Snapshots	733.4 MB RANK 4	269 Threads RANK 4

Table 3: Physical application bundle storage footprint and operating system thread distribution.

5. Autonomous Agent Context Economics: Empirical TokenMem v3 Evaluation

In multi-turn autonomous software engineering tasks, model performance degrades rapidly as context window consumption accumulates. Long prompts induce severe latency penalties, exponential token billing costs, and attention dispersion across hundreds of files.

To empirically evaluate context efficiency under authentic conditions, we executed a standard 15-turn multi-file refactoring scenario comprising 52 discrete tool invocations on the 1.17 GB workspace. BPE token counts were calculated across real source files:

Turn 15 Context Component	Unoptimized Baseline Raw Dumps + Full History	MicroCode TokenMem v3 6-Layer Context Compression	Observed Reduction Ratio
Tool JSON Schemas (52 vs. 9)	16,432 tokens	2,844 tokens	**−82.7%** (Layer 1 Scoping)
Multi-File Code Ingestion (8 Files)	42,314 tokens	3,755 tokens	**−91.1%** (Layer 3 Skeleton)
Terminal & Compiler Logs	14,758 tokens	2,041 tokens	**−86.2%** (Layer 2 Masking)
Turn-15 Single-Prompt Input Tokens	73,504 tokens	**8,640 tokens**	**−88.2% (8.51× Reduction)**
15-Turn Cumulative Input Tokens	654,185 tokens	**77,760 tokens**	**−88.1% (576,425 tokens saved)**

Table 4: BPE token consumption comparison on authentic 1.17 GB repository files across a 15-turn task.

5.1 The Six Pipelined Layers of TokenMem v3

Layer 1: Intent-Driven Tool Schema Scoping `−82.7%`

Filters out inactive tool JSON schemas dynamically based on user engineering intent, reducing schema overhead from 16,432 to 2,844 tokens.

Layer 2: Observation Masking & Decay `−86.2%`

Condenses multi-thousand-line compiler logs into structured diagnostic summaries, preserving only active `error:` and `warning:` coordinates.

Layer 3: AST Skeletonization via Tree-sitter `−91.1%`

Extracts structural type interfaces, protocols, and function signatures while eliding implementation bodies for background reference files.

Layer 4: Rolling Key-Facts Ledger `580 TOKENS`

Encapsulates project constraints and completed decisions into an immutable `StructuredStateCheckpoint`, preventing context drift across turns.

Layer 5: Content-Hash Delta Read Cache `−92.4%`

Maintains SHA-256 digests of previously read buffers; unchanged files emit 290-token pointer references rather than full redundant payloads.

Layer 6: Prompt Prefix Cache Pinning `84.0% HIT`

Enforces byte-level invariance for system instructions and tool schemas, maximizing upstream LLM KV-cache prompt hit rates.

> **Economic Impact on Engineering Operations:** On frontier reasoning models (e.g., Claude 3.7 Sonnet / Gemini 2.5 Pro at $3.00 per million input tokens), a 15-turn unoptimized session costs approximately $1.96 per engineering task. Under TokenMem v3, cumulative tokens drop from 654,185 to 77,760, reducing direct API cost to **$0.23 (8.51× cost reduction)** while cutting inference time-to-first-token (TTFT) by over 70%.

6. Engineering Audit: Root-Cause Analysis & v1.0 Fixes

In accordance with strict scientific integrity, Table 5 documents the architectural discrepancies identified in benchmark suite v1.0 and the verified production engineering resolutions deployed to eliminate all unfair bias:

Identified Discrepancy	Pre-Audit State (v1.0 Suite)	Verified Resolution (v2.5.32 Deployment)
Frame-0 Blank View Artifact	`ContentView` deferred workspace mounting behind `isWorkspaceMounted = false`, stopping WindowServer timers before UI initialization.	Re-architected `ContentView` to mount `AgenticEditorWorkspace` synchronously on frame 0, rendering sidebar and editor immediately.
CLI Single-File Break Loop	Command-line argument loop in `MicroCodeApp.swift` broke after the first argument, only opening the primary file and skipping the root workspace.	Implemented `openLocalLaunchTargets` to ingest both `<workspace_dir>` and `<primary_file>` concurrently, matching Zed and VS Code.
Background Process RAM Spike	`LocalEcosystemDiscovery` spawned `agy models` at *T*+3.5s, invoking an external Electron CLI that bloated RAM to 914.2 MB (105 threads).	Restricted background CLI discovery to cached model manifests during passive startup, stabilizing steady-state RAM at **194.8 MB**.
Duplicate Workspace Reload	`loadSettings()` executed an unconditioned 0.35s timer that reloaded `lastWorkspacePath` over the active CLI session.	Added strict guard `self.workspaceFolder == nil` in `AppState.swift`, reducing workspace ready latency by 206 ms.

Table 5: Comprehensive audit of pre-existing loopholes and verified production resolutions in MicroCode v2.5.32.

7. Engineering Roadmap towards Sub-600ms Cold Startup

While MicroCode v2.5.32 leads in resident memory (194.8 MB, Rank #1), thread count (14 threads, Rank #1), and context optimization, it trails Zed in initial window render by 107.1 ms (769.9 ms vs. 662.8 ms). To achieve Rank #1 across all cold startup metrics, the following optimizations are scheduled for v2.5.33:

Whole-Module Optimization (Release WMO + LTO)
Deploying full Swift compiler Whole-Module Optimization with dead-strip linkage is projected to reduce AppKit view tree compilation time by 140–180 ms.

Early CLI Argument Traversal in App.init()
Initiating asynchronous directory enumeration and zero-copy mmap staging inside `App.init()` prior to `WindowGroup` creation will eliminate the delta between T_{win} and T_{ready}.

8. Data Availability & Certification Statement

All benchmark harnesses (`benchmarks/run_ide_benchmarks.py`), raw telemetry JSON datasets (`benchmarks/results/ide_benchmark_results.json`), and source trees are committed to the official repository for 100% independent replication.

OFFICIAL CERTIFICATION OF ENGINEERING REPRODUCIBILITY

I hereby certify that the experimental data, kernel traces, and comparative metrics presented in this technical report were obtained under 100% identical hardware, OS, and workspace conditions without synthetic bias or privileged instrumentation.

Tirawat Nantamas
Founder & Chief Technology Officer • Dotmini Software Co., Ltd.

Date: October 26, 2026
Identifier: `MC-BENCH-PROD-2026-V2.5.32-GB`